

软件过程改进方案设计文档

课程：软件过程与管理

作业编号：Assgt_02 — 交付物二

项目名称：酒店管理系统 (HotelMS)

学生：潘鸿涛 (2023141461078)

日期：2026年5月

1. 改进背景与目标

1.1 改进背景

在《软件过程剪裁与定义》（作业1）中，HotelMS项目定义了一套以Scrum为主体、“前期偏计划、迭代偏敏捷”的混合软件过程，共包含36项活动，覆盖了项目启动、Sprint迭代、测试、部署和管理五大阶段。

然而，原过程中所有关键活动均依赖人工执行，存在以下瓶颈：

- 需求阶段**：酒店业务规则繁多，用户故事编写耗时且验收标准常不完整
- 设计阶段**：房态状态机设计依赖个人经验，缺乏自动化的完备性验证
- 编码阶段**：CRUD样板代码重复劳动多，人工代码审查的覆盖率和一致性有限
- 测试阶段**：组合场景的测试用例编写工作量大，边界条件容易遗漏

《AI技术应用场景匹配报告》（交付物一）已识别出4个高价值AI应用场景。本文档在此基础上，设计AI技术与原有过程的**深度融合方案**。

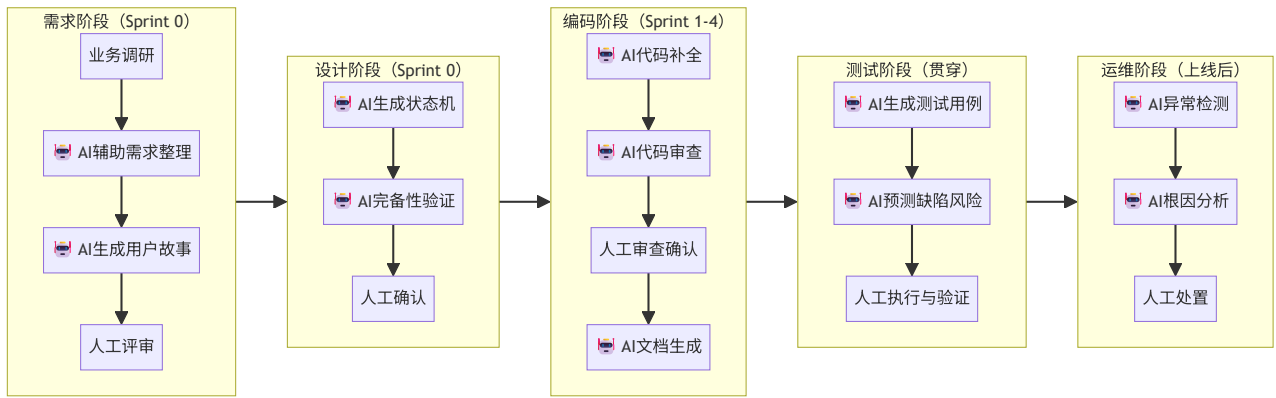
1.2 改进目标

目标维度	现状	改进目标	衡量指标
需求质量	手动编写，验收标准遗漏率约25%	AI辅助生成，遗漏率降至≤10%	评审发现的问题数
开发效率	CRUD模块编码速度约200行/天	AI辅助后提升至300行/天	功能点交付速度
测试覆盖	边界条件覆盖率约60%	AI辅助后提升至≥85%	测试用例覆盖率
缺陷逃逸	P1/P2缺陷逃逸率约15%	AI辅助审查后降至≤8%	上线后缺陷数
文档完整	API文档维护滞后	AI自动生成，实时同步	文档与代码的一致性

2. 过程重构方案

2.1 改进概览：AI融合全景图

在原过程的五个阶段中，AI将在以下节点深度参与：



图例：🤖 标记表示 AI 参与或主导的活动节点

2.2 各阶段过程重构详述

2.2.1 需求分析阶段：引入AI需求助手

原有过程（改进前）：

ACT-03 需求研讨会（人工） → ACT-04 领域建模（人工） → ACT-05 编写用户故事（人工）

改进后过程：

ACT-03 需求研讨会（人工主导 + AI记录） → NEW-AI01 AI需求整理与结构化 → ACT-04 领域建模（

新增活动详述：

新活动ID	活动名称	描述	AI工具	输入 → 输出
NEW-AI01	AI需求整理与结构化	将需求研讨会的录音转文字，AI自动提取关键需求点，按FURPS+（功能性/可用性/可靠性/性能/可支持性）分类整理	语音转文字 + LLM摘要（如讯飞听见 + Claude)	会议录音 → 结构化需求清单
NEW-AI02	AI生成用户故事	输入结构化需求，AI自动生成"作为<角色>，我想要<功能>，以便<价值>"格式的用户故事，每个故事附带AC（验收标准）和边界条件	LLM（Claude/GPT-4o)	需求清单 → 带AC的用户故事集

原有活动调整：

- ACT-05 "编写用户故事"的角色从"全团队创作"变为"全团队评审AI生成的初稿"——从创作型工作转变为评审型工作

2.2.2 系统设计阶段：引入AI建模助手

原有过程（改进前）：

ACT-04 领域建模（人工手绘ER图和状态图） → ACT-07 架构设计（人工）

改进后过程：

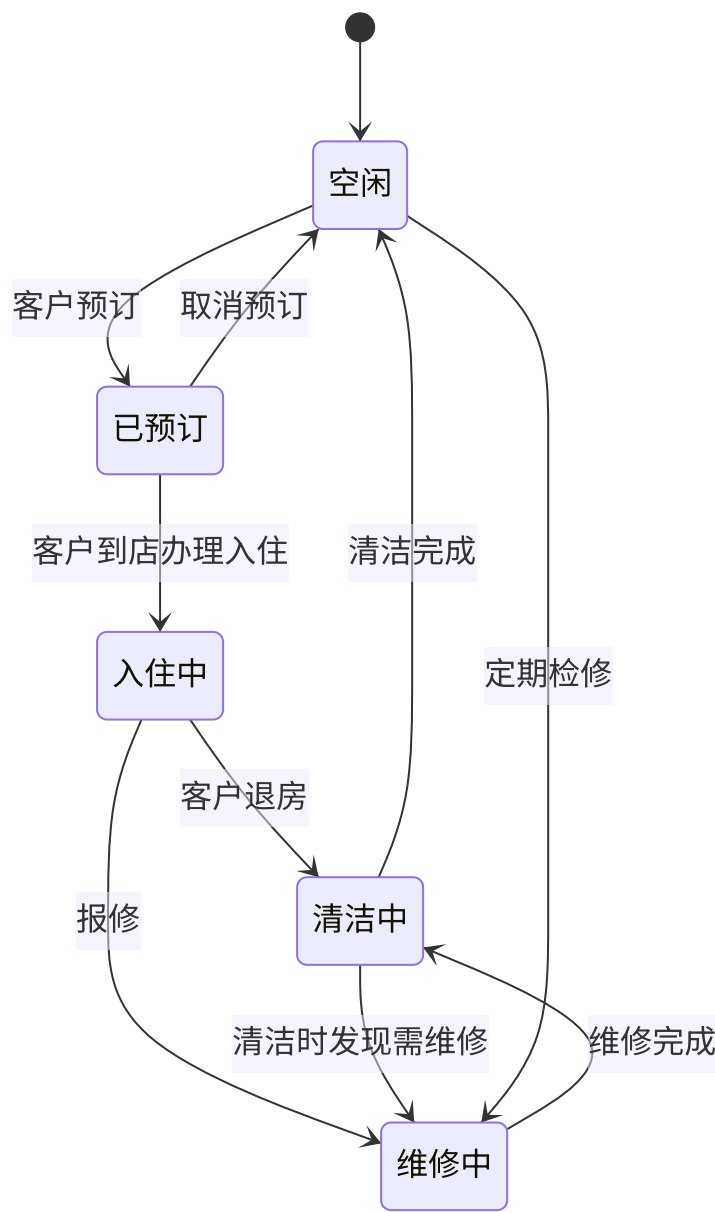
ACT-04 领域建模（人工定义实体 + AI生成ER图） → NEW-AI03 AI生成状态机图 → NEW-AI04 AI状态

新增活动详述：

新活动ID	活动名称	描述	AI工具	输入 → 输出
NEW-AI03	AI生成状态机图	从需求文档中的房态描述文本，AI自动生成Mermaid格式的状态转换图，覆盖所有合法状态转换路径	LLM + Mermaid渲染	需求文本 → 状态机Mermaid图
NEW-AI04	AI状态机完备性验证	AI自动检查状态机中的"死状态"（无法到达的状态）、"黑状态"（无法退出的状态）、"缺失转换"（应该存在但未定义的转换）	LLM逻辑验证	状态机图 → 完备性验证报告

改进案例——房态状态机：

以下是AI根据需求文本自动生成的酒店房态状态机图：



AI同时生成的完备性验证报告：

-  所有5个状态均有入边和出边（无死状态/黑状态）
-  缺少"已预订→维修中"的转换（酒店可能在预订后因房间故障调整房态）
-  缺少"维修中→空闲"的直接转换（轻微维修后可能无需清洁）

2.2.3 编码实现阶段：引入AI编程助手

原有过程（改进前）：

ACT-14 编码实现（人工手写） → ACT-16 代码审查（人工Peer Review）

改进后过程：

NEW-AI05 AI代码生成辅助 → ACT-14 编码实现（人机协同） → ACT-15 单元测试 → ACT-16 代码审查

新增活动详述：

新活动ID	活动名称	描述	AI工具	输入 → 输出
NEW-AI05	AI代码生成辅助	开发者用自然语言描述功能逻辑（如"实现根据会员等级计算折扣的方法"），AI生成代码骨架或完整实现，开发者审查后采纳或修改	GitHub Copilot / Cursor	注释/需求描述 → 代码实现
NEW-AI06	AI生成API文档	每次代码提交后，AI自动扫描Controller层代码，更新Swagger/OpenAPI文档	LLM + Swagger Parser	源代码 → API文档

原有活动调整：

- ACT-16（代码审查）：改为**双层审查机制**——AI先进行自动化审查（安全漏洞、代码规范、房态转换合法性），AI审查通过后再由人工审查（关注业务逻辑和架构一致性）。AI审查结果作为人工审查的参考输入。

AI代码审查检查清单（自动执行）：

检查维度	具体规则	严重级别
安全	身份证号/手机号是否脱敏（日志中和API响应中）	 P0

检查维度	具体规则	严重级别
安全	是否存在SQL拼接（必须用参数化查询）	● P0
安全	敏感接口是否有权限注解（@PreAuthorize）	● P0
业务规则	房态转换是否符合状态机定义	● P1
业务规则	结账金额计算是否包含所有费用项	● P1
代码规范	是否符合阿里巴巴Java开发手册	● P2
代码规范	是否有未处理的异常	● P1

2.2.4 测试验证阶段：引入AI测试助手

原有过程（改进前）：

ACT-22 功能测试（人工编写用例 → 人工执行） → ACT-23 性能测试 → ACT-24 安全测试

改进后过程：

NEW-AI07 AI生成测试用例 → NEW-AI08 AI缺陷风险预测 → ACT-22 功能测试（人工执行 + AI用例补

新增活动详述：

新活动ID	活动名称	描述	AI工具	输入 → 输出
NEW-AI07	AI生成测试用例	读取用户故事的AC和源代码，AI自动生成正向、逆向、边界、异常四类测试用例及测试数据	LLM + 测试框架集成	用户故事 + 源码 → 测试用例集
NEW-AI08	AI缺陷风险预测	基于代码变更量、历史Bug密度、圈复杂度等特征，AI预测当前Sprint中哪些模块最可能出Bug，指导测试优先级	机器学习分类模型	代码变更数据 → 风险热力图
NEW-AI09	AI测试报告生成	测试完成后，AI自动汇总测试结果，生成格式化的测试报告（含通过率、缺陷分布、与上Sprint对比）	LLM	测试执行记录 → 测试报告

测试用例AI生成示例——酒店结账场景：

输入：用户故事"作为前台，我要为退房客人结算费用，以便完成退房流程"

- AC1：系统自动计算房费（含超时加收）
- AC2：系统汇总客人消费（迷你吧、商品等）
- AC3：系统根据会员等级自动计算折扣
- AC4：支持多种支付方式（现金/刷卡/微信/支付宝）

AI生成的测试用例（部分）：

- ✔ 正向用例 TC01：标准退房—住1晚，无额外消费，会员银卡95折，微信支付
- ✔ 正向用例 TC02：延住退房—住3晚，第3天16:00退房，超时4小时，加收半天房费
- ✔ 逆向用例 TC03：退房时房间有未结算迷你吧消费，系统提示先结算
- ✔ 边界用例 TC04：闰年2月28日入住，3月1日退房，房费计算是否正确跨越月尾
- ✔ 边界用例 TC05：会员金卡折扣88折 + 促销活动95折，验证折上折是否按规则叠加
- ✔ 异常用例 TC06：支付过程中第三方支付网关超时，验证回滚和重试机制

2.2.5 运维监控阶段：引入AI运维助手

改进后新增活动：

新活动ID	活动名称	描述	AI工具	输入 → 输出
NEW-AI10	AI异常检测与告警	监控系统指标（QPS、响应时间、错误率），AI自动识别异常模式，减少误报	机器学习异常检测	时序指标 → 异常告警
NEW-AI11	AI根因分析	当故障发生时，AI自动关联日志、链路和事件，快速定位根因	LLM + 可观测性平台	告警+日志 → 根因推断

2.3 新增AI相关角色

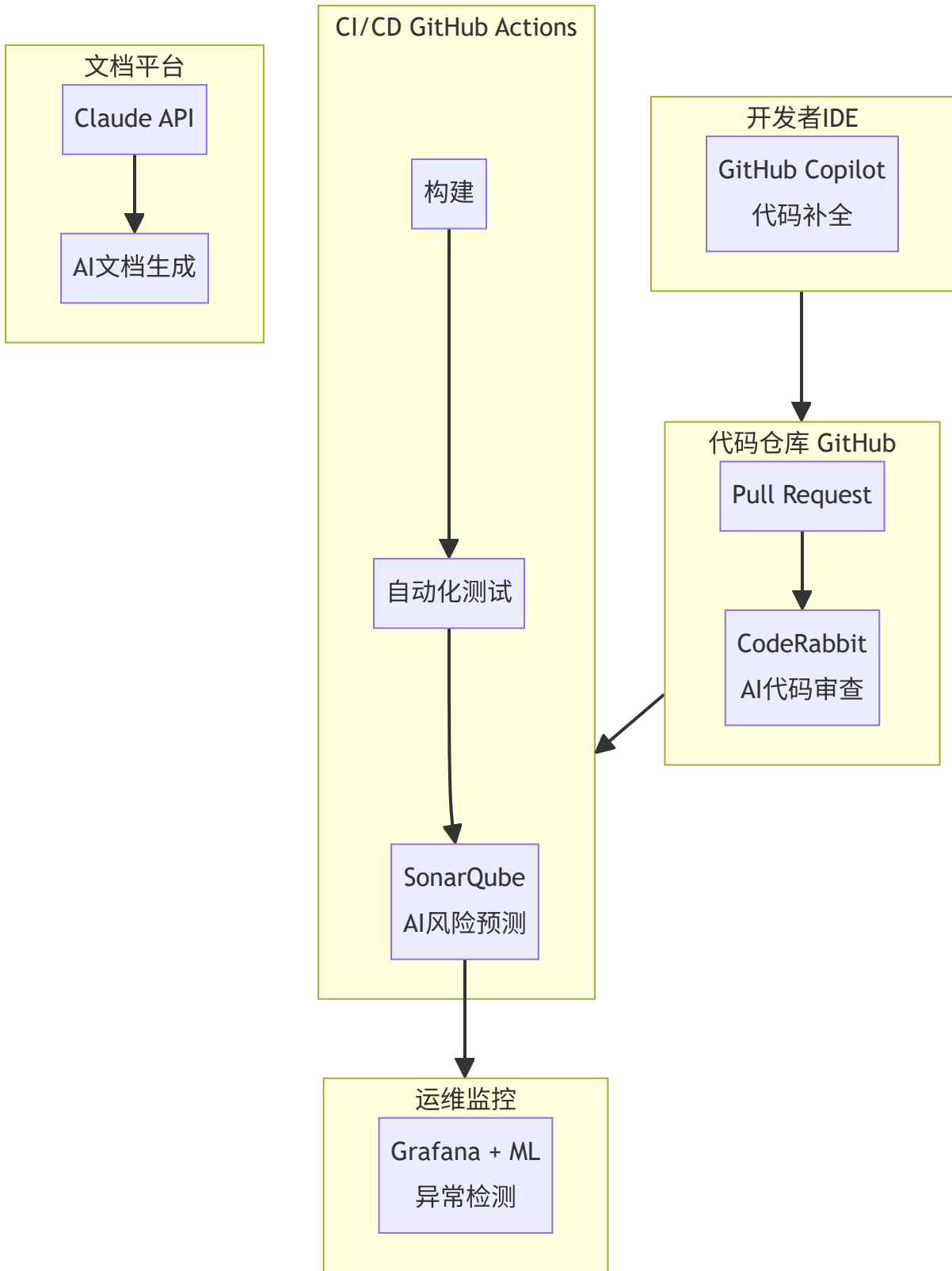
角色	担任方式	职责
AI工具管理员	测试兼运维工程师兼任（新增）	管理AI工具的许可证和配置；维护Prompt模板库；评估新的AI工具并推动试用
AI输出审核人	各阶段的负责人兼任（新增职责）	审查AI生成的需求文档/设计图/代码/测试用例的质量；判断AI输出的采纳/修改/废弃
AI训练师（轻量）	技术负责人兼任（新增）	维护项目级的Prompt工程模板；积累项目专属的AI使用最佳实践；管理"提示词库"

3. 工具链选型

3.1 推荐工具清单

阶段	工具名称	核心功能	选型理由	集成方式	成本
需求	Claude (Anthropic)	需求结构化、用户故事生成、AC自动生成	中文理解能力强，支持长上下文（200K tokens），可一次输入大量酒店业务规则	通过API或Web端交互	\$20/月 (Pro)
需求	讯飞听见	会议语音转文字	中文识别准确率行业领先	独立使用，输出文本→Claude	按量计费
设计	Claude + Mermaid	文本描述自动生成Mermaid状态图	无需额外安装工具，直接在Markdown中渲染	Mermaid代码嵌入Markdown文档	已有
编码	GitHub Copilot	IDE内联代码补全与生成	业界最成熟的AI编程助手，与VS Code/IntelliJ深度集成	IDE插件（VS Code）	\$10/月 (Individual)
编码	CodeRabbit	AI驱动的PR代码审查	自动检测安全漏洞、代码异味、逻辑错误；支持中文注释	GitHub App（PR触发）	免费版可用
测试	Claude	从AC自动生成测试用例和测试数据	可一次性分析多个Story生成全量测试矩阵	API调用	已有
测试	SonarQube + AI	代码质量 + AI风险预测	已在原过程使用，AI增强版支持缺陷预测	CI流水线集成	社区版免费
运维	Grafana + ML插件	AI异常检测	开源方案，成本可控	指标采集 + 面板配置	免费

3.2 工具链集成架构



4. 风险评估与应对策略

4.1 风险识别矩阵

风险ID	风险类别	风险描述	概率	影响	风险等级
R-01	数据安全	AI工具（如Claude API、Copilot）可能将代码或需求文本传输至云端，酒店住客的身份证号、信用卡信息等敏感数据有泄露风险	中	极高	高
R-02	模型偏见	AI生成的代码/测试用例可能偏向某些模式，忽略非主流但重要的场景（如中国特有的农历节日房价策略）	中	中	中
R-03	技术依赖	团队成员过度依赖AI输出，导致自身编码和设计能力退化；AI工具服务中断时团队效率骤降	中	高	中
R-04	输出质量	AI生成的代码/测试用例/文档可能存在微妙的逻辑错误（hallucination），不经审查直接使用可能引入缺陷	高	高	高
R-05	成本失控	AI工具的API调用费、许可证费超出预算	低	中	低
R-06	合规风险	使用境外AI服务商（OpenAI/Anthropic）处理酒店经营数据，可能违反《个人信息保护法》的数据出境规定	中	极高	高

4.2 风险应对策略

R-01 & R-06：数据安全和合规

应对策略	具体措施
数据脱敏 (首选)	在将数据送入AI工具前，对所有敏感字段（姓名、身份证号、手机号、银行卡号）进行脱敏处理，替换为模拟数据（如"张三"→"张**"，"510xxx19900101xxxx"→"000000199001010000"）
工具使用规范	制定《AI工具使用规范》，明确：禁止向AI工具提交真实客户数据；代码审查时如发现代码中包含硬编码的敏感数据，AI审查必须告警
本地化部署 (长期)	评估使用私有化部署的开源模型（如DeepSeek、Qwen），在本地服务器运行，数据不出企业网络
合规审查	在Sprint 0中增加"AI工具合规审查"节点，确认每个选型工具的数据处理协议符合合规要求

R-02：模型偏见

应对策略	具体措施
多样化 Prompt	在Prompt中显式要求AI"考虑中国酒店行业的特殊场景：农历节假日、钟点房、长包房、旅行团协议价等"
人工校验清单	为AI输出审核人建立检查清单，明确列出"AI容易忽略的中国特有场景"
多模型交叉验证	对关键输出（如房价计算逻辑），用两个不同的AI模型交叉验证

R-03：技术依赖

应对策略	具体措施
人工先行原则	规定：开发者必须先手动理解问题并写出核心逻辑思路，然后才能使用AI辅助（非简单粘贴AI生成的代码）
技能保持机制	每个Sprint至少分配一个Story完全人工完成（不借助AI），作为技能保持的"锚点"
故障演练	模拟"AI服务不可用"场景，验证团队是否能在无AI辅助的情况下维持基本生产效率

R-04：输出质量

应对策略	具体措施
双层审查机制（已融入过程）	AI输出物（需求/设计/代码/测试用例）必须经人工审查确认后才能合并或发布
AI输出标记	AI生成的代码块要求用注释标记 <code>// [AI-Generated, Reviewed]</code> ，AI生成的测试用例标记数据来源标签
质量度量	跟踪"AI生成内容的采纳率"和"AI生成内容的缺陷率"两个指标，评估AI输出的可靠性

R-05：成本失控

应对策略	具体措施
用量监控	设定API调用量月度上限，达到80%时预警
优先级分配	AI工具优先用于P0/P1场景（需求和测试），CRUD代码生成可用免费层级的Copilot

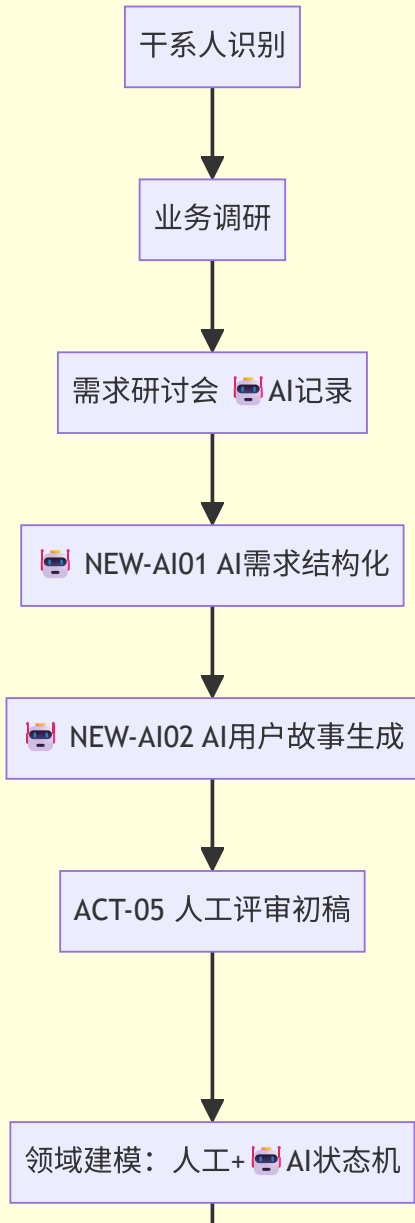
5. 改进后过程度量体系

在原有9项度量指标的基础上，新增AI贡献度相关指标：

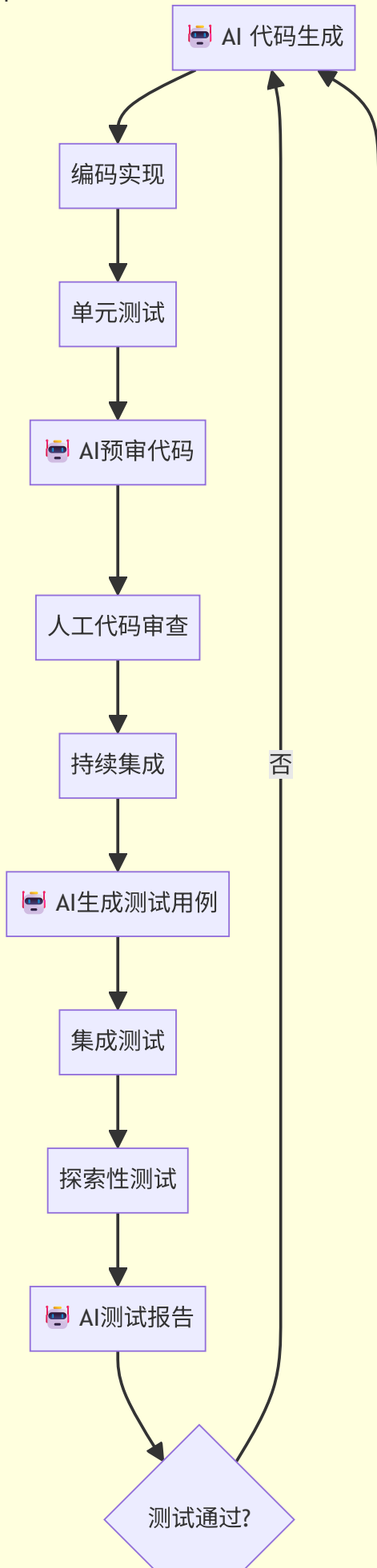
度量 维度	新增指标	目标值	说明
AI效率	AI代码采纳率	60-80%	(采纳的AI生成代码行数 / AI生成的代码总行数)；过低说明AI输出质量差，过高说明缺乏人工审查
AI效率	AI测试用例有效率	≥70%	(AI生成且人工确认有效的测试用例数 / AI生成的测试用例总数)
AI质量	AI输出缺陷率	≤5%	(AI生成内容中经人工审查发现缺陷的比例)
AI覆盖	AI参与活动占比	≥40%	(有AI参与的活动数 / 过程总活动数)，监控AI渗透率
风险	AI工具可用率	≥99%	AI工具月度正常服务时间占比

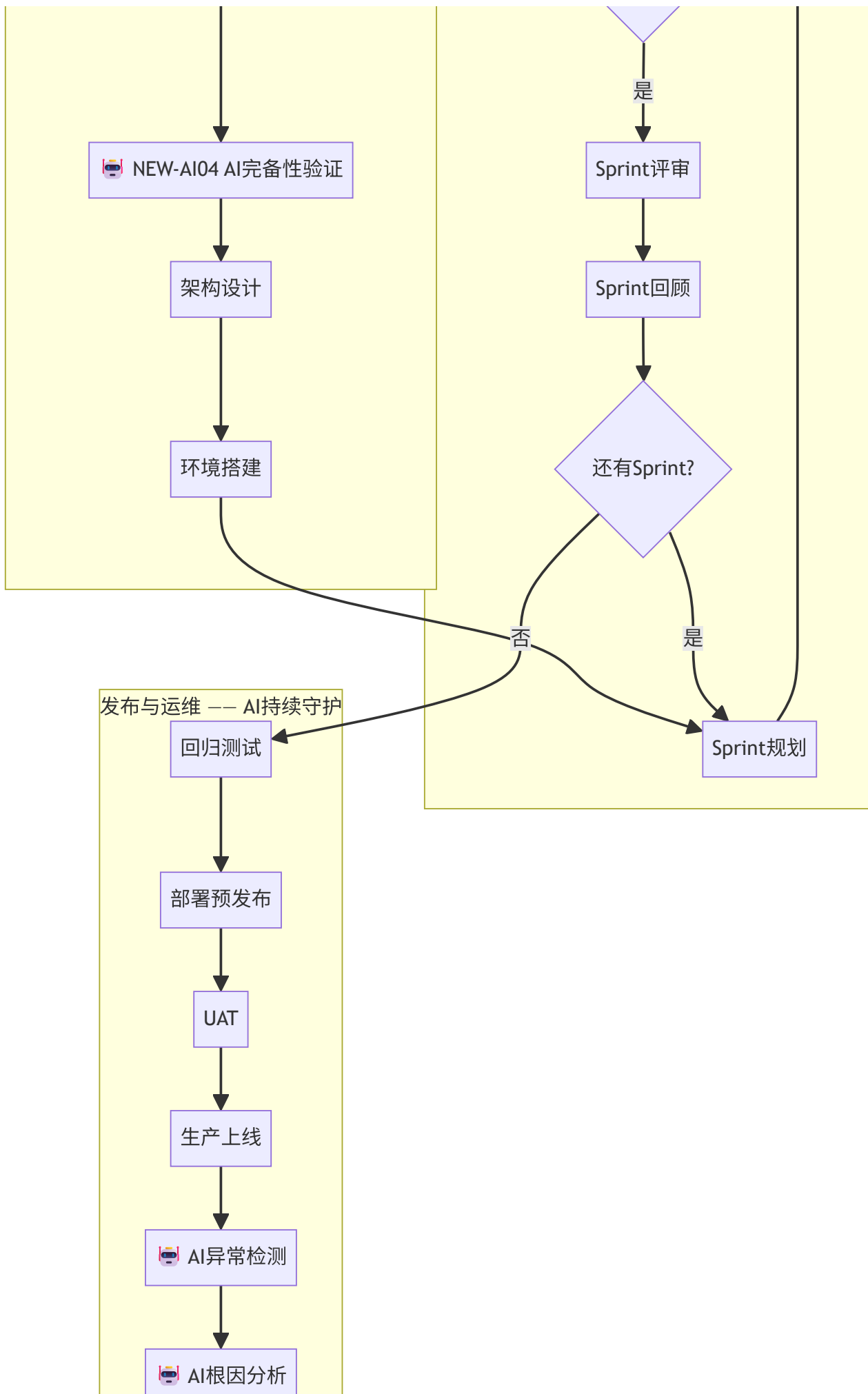
6. 改进后完整流程图

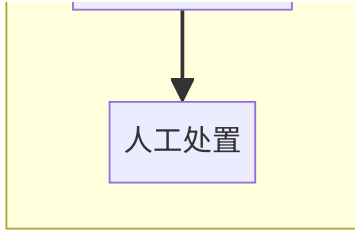
Sprint 0: 启动与奠基（第1-2周）—— AI深度融合



开发Sprint循环（×4次，每次3周）—— AI持续辅助







参考文献

1. GitHub. (2025). GitHub Copilot: 2025 Impact and Adoption Report
2. CodeRabbit. (2025). AI-Powered Code Review: Best Practices Guide
3. Anthropic. (2025). Claude for Software Engineering: Prompt Engineering Guide
4. 中国信通院. (2024). 《智能化软件工程技术和应用要求 第3部分：智能测试能力》
5. Google. (2025). Predictive Defect Detection with Machine Learning at Scale
6. PagerDuty. (2025). AIOps Maturity Model and Implementation Guide
7. SonarSource. (2025). SonarQube 2025: AI-Enhanced Code Quality Analysis